

IFT3295 — Démo 2 : correction

18 septembre 2026

Score d'un alignement de paire de séquences : pénalité de gaps

La pondération des gaps influence grandement l'alignement entre deux séquences. Certaines pondérations ont parfois plus de sens biologique que d'autres. Soit l'alignement A suivant :

S: ACGTTTACG-TAC-T
T: ACC---ACGCTAGCT

En considérant les scores suivants :

- match : $+1$
- mismatch : -1
- gap : -1

Donnez les scores de l'alignement A pour chacune des pondérations de gaps suivantes :

1. pondération constante
2. pondération linéaire (utiliser un score de -1 pour un indel)
3. pondération affine (utiliser une extension de gap $\sigma = -0,5$)
4. pondération convexe (utiliser un score $-1 - \ln(t)$)

Réponse :

L'alignement A contient 8 matchs, 2 mismatches et 3 blocs de gaps (5 indels).

Variables :

- n : nombre d'occurrences d'un événement
- mat : diminutif de « match »
- mis : diminutif de « mismatch »
- c_{gaps} : coût total des gaps (dépend de la pondération choisie)

La formule générale dans ce contexte est donc :

$$Score = n_{mat} \cdot score_{mat} + n_{mis} \cdot score_{mis} + c_{gaps}$$

avec $n_{mat} = 8$, $n_{mis} = 2$, $n_{blocs} = 3$ et $n_{indels} = 5$.

1. $Score_{constante} = 8 \cdot 1 + 2 \cdot (-1) + 3 \cdot (-1) = 3$ (une colonne de gaps coûte -1 , quelle que soit sa longueur)
2. $Score_{lineaire} = 8 \cdot 1 + 2 \cdot (-1) + 5 \cdot (-1) = 1$ (chaque indel coûte -1)
3. $Score_{affine} = 8 \cdot 1 + 2 \cdot (-1) + 3 \cdot (-1) + 5 \cdot (-0,5) = 0,5$
Chaque bloc paie une ouverture de -1 et **chaque** indel du bloc paie l'extension $\sigma = -0,5$: les trois blocs (longueurs 3, 1 et 1) coûtent au total $3 \cdot (-1) + 5 \cdot (-0,5)$.
4. $Score_{convexe} = 8 \cdot 1 + 2 \cdot (-1) + (-1 - \ln 3) + (-1 - \ln 1) + (-1 - \ln 1) \approx 1,90$

Programmation dynamique et alignement de séquences

1. Alignement avec distance d'édition
 - (a) Donner les équations de récurrences ainsi que les conditions initiales pour construire la table de programmation dynamique pour l'alignement entre deux séquences S et T avec la distance d'édition.

Réponse :

Variables : $D(i, j)$ = distance d'édition entre $S[1..i]$ (i premières lettres de S , en lignes) et $T[1..j]$ (en colonnes).

Conditions initiales : $D(i, 0) = i$ et $D(0, j) = j$

Réurrence :

$$D(i, j) = \min \begin{cases} D(i, j-1) + \delta(-, t_j) \\ D(i-1, j) + \delta(s_i, -) \\ D(i-1, j-1) + \delta(s_i, t_j) \end{cases}$$

Remarque : bien que la distance d'édition soit composée d'insertions, de délétions et de substitutions, on peut modéliser celles-ci de la même façon en utilisant des gaps. Un mismatch correspond à une substitution et les indels correspondent à des insertions ou délétions.

- (b) Construire la table de programmation dynamique pour les deux séquences suivantes : $S = \text{CTGCGACA}$ et $T = \text{TGCAAG}$

Réponse : la ligne i se remplit avec les trois voisins (haut +1, gauche +1, diagonale +0 si match, +1 sinon).

D		T	G	C	A	A	G
	0	1	2	3	4	5	6
C	1	1	2	2	3	4	5
T	2	1	2	3	3	4	5
G	3	2	1	2	3	4	4
C	4	3	2	1	2	3	4
G	5	4	3	2	2	3	3
A	6	5	4	3	2	2	3
C	7	6	5	4	3	3	3
A	8	7	6	5	4	3	4

$D(8, 6) = 4$.

Lecture des flèches : en **rouge gras**, le déplacement suivi par le traceback (la solution choisie) ; en **bleu**, toutes les flèches des parcours optimaux qui partent de la case de réponse (elles mènent à une autre solution optimale de même coût) ; en noir, les flèches qui ne font partie d'aucun de ces parcours.

- (c) Comment peut-on retrouver un alignement optimal de séquence à partir de la table ?

Réponse :

Il faut suivre les pointeurs de la case en bas à droite de la table jusqu'à la case $(0, 0)$: à chaque case, remonter vers le voisin (haut, gauche ou diagonale) qui explique la valeur. Ici, un alignement optimal de coût 4 :

C	T	G	C	G	A	C	A
-	T	G	C	-	A	A	G

- (d) Donner la complexité de l'algorithme d'alignement de paires de séquences avec la distance d'édition.

Réponse :

- En temps : $\mathcal{O}(mn)$ où m est la taille de T et n est la taille de S car nous devons remplir une table de taille $m \times n$ et chaque cellule nécessite un nombre constant d'opérations pour être calculée.
- En espace : $\mathcal{O}(mn)$ pour stocker la table de programmation dynamique.

2. Que faut-il changer si on désire utiliser la similarité plutôt qu'une mesure de distance pour aligner deux séquences ?

Réponse :

1. Il faut changer la fonction de score pour qu'elle maximise la similarité au lieu de minimiser la distance (matches de score positif, mismatches et gaps de score négatif).
2. Il faut changer les conditions initiales : au lieu de $D(i, 0) = i$ et $D(0, j) = j$, les bords *cumulent* les scores d'appariement, c.-à-d. que chaque gap de bord paie son score (initiales cumulées, comme pour Needleman-Wunsch).

3. Alignement local vs. global

- (a) Quelle est la différence entre un alignement local et un alignement global ?

Réponse :

Un alignement global est un alignement des deux séquences au complet, alors qu'un alignement local est un alignement d'un facteur d'une séquence avec un facteur de l'autre séquence.

- (b) Dans le cas d'un alignement local, la distance d'édition est-elle utile ? Justifiez

Réponse :

Non ! La distance d'édition n'est pas utile, car il existe toujours un alignement local « vide » de distance 0 quelles que soient les séquences en entrée.

- (c) Donnez les conditions initiales et les équations de récurrence de l'alignement local Smith-Waterman et de l'alignement global Needleman-Wunsch.

Réponse :

Variables :

$V(i, j)$: score de l'alignement max de $s[1..i]$, $t[1..j]$

$\delta(a, b)$: score de l'appariement de a et b

Needleman-Wunsch

Conditions initiales :

$$V(i, 0) = \sum_{1 \leq k \leq i} \delta(s_k, -) \text{ et } V(0, j) = \sum_{1 \leq k \leq j} \delta(-, t_k)$$

Relation de récurrence :

$$V(i, j) = \max \begin{cases} V(i, j-1) + \delta(-, t_j) \\ V(i-1, j) + \delta(s_i, -) \\ V(i-1, j-1) + \delta(s_i, t_j) \end{cases}$$

Smith-Waterman

Conditions initiales :

$$V(i, 0) = 0 \text{ et } V(0, j) = 0$$

Relation de récurrence :

$$V(i, j) = \max \begin{cases} 0 & \text{parcours gratuit} \\ V(i, j-1) + \delta(-, t_j) \\ V(i-1, j) + \delta(s_i, -) \\ V(i-1, j-1) + \delta(s_i, t_j) \end{cases}$$

4. Recherche de motif : chercher $P = \text{GCG}$ dans $T = \text{CTGCGCACA}$ (à k erreurs près).

- (a) Quels scores d'appariement et quelles conditions initiales faut-il choisir pour laisser l'alignement libre sur T tout en alignant P en entier ?

Réponse : nous utilisons la distance d'édition mais avec des conditions initiales modifiées : alignement libre sur la séquence T , mais P aligné en entier.

Variables : $D(i, j)$: distance d'édition entre $P[1..i]$ et $T[1..j]$;
 $\delta(a, b)$: coût de l'appariement de a et b .

Scores :

match : 0

mismatch : 1

gap : 1

Conditions initiales : $D(0, j) = 0$ pour $0 \leq j \leq m$ (départ libre dans T) et $D(i, 0) = i$ pour $1 \leq i \leq n$ (P s'aligne en entier).

- (b) Donnez la relation de récurrence de la table $D(i, j)$.

Réponse :

$$D(i, j) = \min \begin{cases} D(i, j-1) + \delta(-, t_j) \\ D(i-1, j) + \delta(s_i, -) \\ D(i-1, j-1) + \delta(s_i, t_j) \end{cases}$$

- (c) Construire la table de programmation dynamique.

Réponse :

D		C	T	G	C	G	C	A	C	A
	0	0	0	0	0	0	0	0	0	0
G	1	1	1	0	1	0	1	1	1	1
C	2	1	2	1	0	1	0	1	1	2
G	3	2	2	2	1	0	1	1	2	2

- (d) Comment retrouve-t-on la position du motif dans T à partir de la table ? Où se termine l'occurrence exacte de **GCG** ?

Réponse : on cherche une case ayant une valeur $\leq k$ dans la

dernière ligne de la table : l'index en j de cette case donne la position de la fin du motif dans T . Ici, la case 0 de la dernière ligne (en vert) donne une occurrence exacte de **GCG** finissant à la position 5 de T ($T[3..5] = \mathbf{GCG}$); toute case $\leq k$ de cette ligne donnerait une occurrence approximative.

5. Problème de la plus longue sous-séquence commune. On utilise les deux séquences suivantes : $S = \text{CTGCGACA}$ et $T = \text{TGCAAG}$.

- (a) Qu'est-ce qu'une sous-séquence, par opposition à une sous-chaîne ? Donnez les scores d'appariement à utiliser.

Réponse : une sous-séquence se lit **dans l'ordre**, sans être forcément contiguë ; une sous-chaîne (facteur) est au contraire **contiguë** (caractères consécutifs). Il existe plusieurs manières de résoudre ce problème en $\mathcal{O}(mn)$; voici une approche utilisant l'alignement global.

Variables : $V(i, j)$: taille de la plus longue sous-séquence commune entre $s[1..i]$ et $t[1..j]$; $\delta(a, b)$: score de l'appariement de a et b .

Scores :

match : +1

gap : 0

mismatch : $-\infty$

- (b) Donnez les conditions initiales et la relation de récurrence.

Réponse : conditions initiales $V(i, 0) = 0$ et $V(0, j) = 0$, et

$$V(i, j) = \max \begin{cases} V(i, j-1) + \delta(-, t_j) \\ V(i-1, j) + \delta(s_i, -) \\ V(i-1, j-1) + \delta(s_i, t_j) \end{cases}$$

- (c) Construire la table de programmation dynamique.

Réponse : chaque case prend le maximum de ses trois voisines ; les gaps gratuits (score 0) sautent les caractères non partagés et le mismatch à $-\infty$ interdit d'apparier des caractères différents : seuls des matches composent l'alignement, et ils lisent exactement une sous-séquence commune.

<i>D</i>		T	G	C	A	A	G
	0	0	0	0	0	0	0
C	0	0	0	1	1	1	1
T	0	1	1	1	1	1	1
G	0	1	2	2	2	2	2
C	0	1	2	3	3	3	3
G	0	1	2	3	3	3	4
A	0	1	2	3	4	4	4
C	0	1	2	3	4	4	4
A	0	1	2	3	4	5	5

- (d) Quelle est la taille de la plus longue sous-séquence commune, et quelle est cette sous-séquence ? Comment la retrouve-t-on dans la table ?

Réponse : $V(n, m)$ est la taille de la plus longue sous-séquence commune ; ici $V(8, 6) = \mathbf{5}$, avec **TGCAA**. On la retrouve en suivant les pointeurs depuis (n, m) jusqu'à $(0, 0)$; ici le chemin optimal est **unique** : chaque case du chemin n'offre qu'un seul successeur optimal, la flèche rouge le trace case par case et aucune flèche bleue n'apparaît.

Remarque : pour la plus longue sous-chaîne commune (facteurs *contigus*), on utilise plutôt un alignement *local* avec mismatches et gaps à $-\infty$. Vous pouvez consulter la page Wikipédia « [Plus longue sous-chaîne commune](#) » pour plus de détails.